TD 03: dynamique du λ-calcul

1 Rappels

La sémantique opérationnelle du λ-calcul sans stratégie est la suivante.

$$\frac{}{(\lambda x.t)\ u \to t\{u/x\}}\ \beta \qquad \frac{t \to t'}{t\ u \to t'\ u}\ \text{APPL} \qquad \frac{u \to u'}{t\ u \to t\ u'}\ \text{APPR} \qquad \frac{t \to t'}{\lambda x.t \to \lambda x.t'}\ \lambda$$

Une forme normale est un terme qui ne peut pas se réduire. Le λ-calcul est confluent, ce qui implique l'unicité de la forme normale (si elle existe).

Les langages de programmation utilise des stratégies de réduction en appel par nom (Haskell) ou par valeur. En appel par nom :

$$\frac{}{(\lambda x.t)\ u \to t\{u/x\}}\ \beta \qquad \frac{t \to t'}{t\ u \to t'\ u}\ \text{APPL}$$

En appel par valeur, de droite à gauche :

$$\frac{}{(\lambda x.t)\ v \to t\{v/x\}}\ \beta \qquad \frac{u \to u'}{t\ u \to t\ u'}\ \text{APPR} \qquad \frac{t \to t'}{t\ v \to t'\ v}\ \text{APPL}$$

Les valeurs v sont soit les variables, soit les abstractions.

2 λ-calcul

Question 1 – Les valeurs (variables et abstractions) sont-elles les seules formes normales du λ-calcul en appel par valeur ?

Question 2 – Évaluer les termes suivants en appel par nom et par valeur, dans lesquels id = λx.x.

1. (λfx.f x) (id id)
2. (λxy.y) id (z id)

Question 3 – Est-ce que les résultats obtenus à la question précédente sont des formes normales du calcul sans stratégie ? Si non, continuer de réduire ces termes.

3 Expressions booléennes

On considère le noyau des expressions booléennes suivant.

EXPRESSIONS

b ::=	`true`	(vrai)	**Priorité** : (negation) > (et) > (ou)
	`false`	(faux)	**Associativité gauche** : et, ou
	b_1 `&&` b_2	(et)	
	b_1 `||` b_2	(ou)	
	`not` b	(negation)	

Question 4 – Quelles devraient être les formes normales de ce calcul ?

La plupart des langages de programmation impératifs évaluent les expressions booléennes de gauche à droite de manière paresseuse :

- si le résultat de l'évaluation de l'expression de gauche d'un (et) est (faux), l'expression de droite n'est pas évaluée, et le résultat du (et) est (faux) ;
- si le résultat de l'évaluation de l'expression de gauche d'un (ou) est (vrai), l'expression de droite n'est pas évaluée, et le résultat du (ou) est (vrai).

Question 5 – Définir la dynamique du langage $b \to b'$ sous forme d'une sémantique opérationnelle structurelle, évaluée de gauche à droite de manière paresseuse.

4 Évaluation et effets de bords en Scala

Le langage de programmation Scala possède deux instructions pour définir des valeurs :

- l'instruction `val i = 3+4` évalue l'expression 3+4, et associe sa valeur au nom `i` ;
- l'instruction `def suc (x: Int) = x+1` définit la fonction `suc` qui est évaluée à chaque appel.

Il est possible de définir des fonctions sans arguments `def hello = println("hello")`. Le corps d'un `val` est évalué *une seule fois*, celui d'une `def` *à chaque appel* à cette `def`. Par exemple, l'exécution de

```
val troisVal = { println("val"); 3} //affiche ‘‘val’’
def troisDef = { println("def"); 3} //n’affiche rien
val res = troisVal + troisDef + troisVal + troisDef //affiche ‘‘def’’ ‘‘def’’
```

affiche `val def def`.

Question 6 – Que voit-on à l'écran à l'éxecution du programme suivant ?

```
val trois = { println("trois"); 3 }
def quatre = { println("quatre"); 4 }
def cinq = {
  val deux = { println("deux"); 2 }
  deux+trois
}
val six = {
  val deux = { println("deux"); 2 }
  deux+quatre
}

println(trois+quatre+cinq+six)
```

Vous pouvez vérifier votre réponse avec un évaluateur en ligne comme `https://scastie.scala-lang.org/`